

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define CREDENTIAL_FILE "credentials.txt"
#define STUDENT_FILE "students.txt"
struct student {
    int roll;
    char name[50];
    float marks;
};
char currentUser[20];
char currentRole[20];
void createCredentials();
int loginSystem();
void mainMenu();
void adminMenu();
void userMenu();
void addStudent();
void displayStudents();
void searchStudent();
void updateStudent();
void deleteStudent();
```

```
int main() {
    createCredentials();
    if (loginSystem()) {
        mainMenu();
    } else {
        printf("Login failed! Exiting...\n");
    }
    return 0;
}

void createCredentials() {
    char choice;
    char user[20], pass[20], role[10];
    printf("Do you want to create login credentials? (y/n): ");
    scanf(" %c", &choice);
    if (choice == 'y' || choice == 'Y') {
        printf("Enter new username: ");
        scanf("%s", user);
        printf("Enter new password: ");
        scanf("%s", pass);
        printf("Enter role (admin/user): ");
        scanf("%s", role);
        FILE *fp = fopen(CREDENTIAL_FILE, "w");
        if (!fp) {
```

```

        printf("Error creating credential file!\n");
        return;
    }

    fprintf(fp, "%s %s %s\n", user, pass, role);
    fclose(fp);

    printf("\nCredentials created successfully!\n");
}

FILE *f = fopen(STUDENT_FILE, "r");
if (!f) {
    f = fopen(STUDENT_FILE, "w");
    fclose(f);
} else {
    fclose(f);
}
}

int loginSystem() {
    char username[20], password[20];
    char fileUser[20], filePass[20], fileRole[20];
    printf("\n===== LOGIN SCREEN =====\n");
    printf("Enter username: ");
    scanf("%s", username);
    printf("Enter password: ");

```

```
scanf("%s", password);
FILE *fp = fopen(CREDENTIAL_FILE, "r");
if (!fp) {
    printf("Credential file missing!\n");
    return 0;
}
while (fscanf(fp, "%s %s %s", fileUser, filePass, fileRole) != EOF) {
    if (strcmp(username, fileUser) == 0 &&
        strcmp(password, filePass) == 0) {
        strcpy(currentUser, fileUser);
        strcpy(currentRole, fileRole);
        fclose(fp);
        return 1;
    }
}
fclose(fp);
return 0;
}

void mainMenu() {
    if (strcmp(currentRole, "admin") == 0)
        adminMenu();
    else
        userMenu();
}
```

```
}  
  
void adminMenu() {  
    int ch;  
    do {  
        printf("\n===== ADMIN MENU =====\n");  
        printf("1. Add Student\n");  
        printf("2. Display Students\n");  
        printf("3. Search Student\n");  
        printf("4. Update Student\n");  
        printf("5. Delete Student\n");  
        printf("6. Logout\n");  
        printf("Enter choice: ");  
        scanf("%d", &ch);  
        switch (ch) {  
            case 1: addStudent(); break;  
            case 2: displayStudents(); break;  
            case 3: searchStudent(); break;  
            case 4: updateStudent(); break;  
            case 5: deleteStudent(); break;  
            case 6: return;  
            default: printf("Invalid choice!\n");  
        }  
    } while (1);  
}
```

```
}  
  
void userMenu() {  
    int ch;  
    do {  
        printf("\n===== USER MENU =====\n");  
        printf("1. Display Students\n");  
        printf("2. Search Student\n");  
        printf("3. Logout\n");  
        printf("Enter choice: ");  
        scanf("%d", &ch);  
        switch (ch) {  
            case 1: displayStudents(); break;  
            case 2: searchStudent(); break;  
            case 3: return;  
            default: printf("Invalid choice!\n");  
        }  
    } while (1);  
}  
  
void addStudent() {  
    struct student st;  
    FILE *fp = fopen(STUDENT_FILE, "a");  
    printf("Enter roll number: ");  
    scanf("%d", &st.roll);
```

```

    printf("Enter name: ");
    scanf("%s", st.name);
    printf("Enter marks: ");
    scanf("%f", &st.marks);
    fprintf(fp, "%d %s %.2f\n", st.roll, st.name, st.marks);
    fclose(fp);
    printf("Student added successfully!\n");
}

void displayStudents() {
    struct student st;
    FILE *fp = fopen(STUDENT_FILE, "r");
    printf("\nRoll\tName\tMarks\n");
    printf("-----\n");
    while (fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) != EOF)
    {
        printf("%d\t%s\t%.2f\n", st.roll, st.name, st.marks);
    }
    fclose(fp);
}

void searchStudent() {
    int roll;
    printf("Enter roll number to search: ");
    scanf("%d", &roll);
    struct student st;

```

```
FILE *fp = fopen(STUDENT_FILE, "r");
int found = 0;
while (fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) != EOF)
{
    if (st.roll == roll) {
        printf("\nFound: %d %s %.2f\n", st.roll, st.name, st.marks);
        found = 1;
        break;
    }
}

if (!found)
    printf("Student not found!\n");

fclose(fp);
}

void updateStudent() {
    int roll;
    printf("Enter roll number to update: ");
    scanf("%d", &roll);
    struct student st;
    FILE *fp = fopen(STUDENT_FILE, "r");
    FILE *tmp = fopen("temp.txt", "w");
    int found = 0;
```



```
while (fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) != EOF)
{
    if (st.roll == roll) {
        found = 1;
        printf("Enter new name: ");
        scanf("%s", st.name);
        printf("Enter new marks: ");
        scanf("%f", &st.marks);
    }
    fprintf(tmp, "%d %s %.2f\n", st.roll, st.name, st.marks);
}

fclose(fp);
fclose(tmp);
if (found) {
    remove(STUDENT_FILE);
    rename("temp.txt", STUDENT_FILE);
    printf("Record updated!\n");
} else {
    remove("temp.txt");
    printf("Roll number not found!\n");
}

}

void deleteStudent() {
    int roll;
```

```
printf("Enter roll number to delete: ");
scanf("%d", &roll);
struct student st;
FILE *fp = fopen(STUDENT_FILE, "r");
FILE *tmp = fopen("temp.txt", "w");
int found = 0;
while (fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) != EOF)
{
    if (st.roll == roll) {
        found = 1;
        continue;
    }

    fprintf(tmp, "%d %s %.2f\n", st.roll, st.name, st.marks);
}

fclose(fp);
fclose(tmp);
if (found) {
    remove(STUDENT_FILE);
    rename("temp.txt", STUDENT_FILE);
    printf("Record deleted!\n");
} else {
    remove("temp.txt");
    printf("Roll number not found!\n");
}
```

```
}  
  
}
```

LOGIN MODULES

```
int loginSystem() {  
    char username[20], password[20];  
    char fileUser[20], filePass[20], fileRole[20];  
  
    printf("\n===== LOGIN SCREEN =====\n");  
    printf("Enter username: ");  
    scanf("%s", username);  
    printf("Enter password: ");  
    scanf("%s", password);  
  
    FILE *fp = fopen(CREDENTIAL_FILE, "r");  
    if (!fp) {  
        printf("Credential file missing!\n");  
        return 0;  
    }  
  
    while (fscanf(fp, "%s %s %s", fileUser, filePass, fileRole) != EOF) {  
        if (strcmp(username, fileUser) == 0 &&  
            strcmp(password, filePass) == 0) {  
  
            strcpy(currentUser, fileUser);  
            strcpy(currentRole, fileRole);  
            fclose(fp);  
            return 1;  
        }  
    }  
  
    fclose(fp);  
    return 0;  
}
```

OUTPUT

```
Do you want to create login credentials? (y/n): y  
Enter new username: HariVenakt  
Enter new password: AP24110011941  
Enter role (admin/user): admin  
  
Credentials created successfully!  
  
===== LOGIN SCREEN =====  
Enter username: HariVenakt  
Enter password: AP24110011941  
  
Login Successful!
```

This module reads the username and password from the user, verifies it against the saved credentials in credentials.txt. If matching, login is successful and role (admin/staff/guest) is stored

MENU BAR

```
void createCredentials() {
    char choice;
    char user[20], pass[20], role[10];

    printf("Do you want to create login credentials? (y/n): ");
    scanf(" %c", &choice);

    if (choice == 'y' || choice == 'Y') {
        printf("Enter new username: ");
        scanf("%s", user);

        printf("Enter new password: ");
        scanf("%s", pass);

        printf("Enter role (admin/user): ");
        scanf("%s", role);

        FILE *fp = fopen(CREDENTIAL_FILE, "w");
        if (!fp) {
            printf("Error creating credential file!\n");
            return;
        }

        fprintf(fp, "%s %s %s\n", user, pass, role);
        fclose(fp);

        printf("\nCredentials created successfully!\n");
    }
}
```

OUTPUT

```
===== ADMIN MENU =====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout
```

ADD STUDENT MODULE

```
void addStudent() {
    struct student st;
    FILE *fp = fopen(STUDENT_FILE, "a");

    printf("Enter roll number: ");
    scanf("%d", &st.roll);

    printf("Enter name: ");
    scanf("%s", st.name);

    printf("Enter marks: ");
    scanf("%f", &st.marks);

    fprintf(fp, "%d %s %.2f\n", st.roll, st.name, st.marks);
    fclose(fp);

    printf("Student added successfully!\n");
}
```

OUTPUT

```
===== ADMIN MENU =====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout
Enter choice: 1
Enter roll number: 2400040375
Enter name: Revanth
Enter marks: 97.5
Student added successfully!
```

Allows admin/staff to insert new student records into the student file

DISPLAY MODULE

```
void displayStudents() {
    struct student st;
    FILE *fp = fopen(STUDENT_FILE, "r");

    printf("\nRoll\tName\tMarks\n");
    printf("-----\n");

    while (fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) != EOF) {
        printf("%d\t%s\t%.2f\n", st.roll, st.name, st.marks);
    }

    fclose(fp);
}
```

OUTPUT

```
===== ADMIN MENU =====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout
Enter choice: 2

Roll      Name      Marks
-----
2         AP400040375    30.00
1         AP2400040305   30.00
204      AP2400040382   85.50
```

This module retrieves all stored student records and displays them in a structured tabular format. It reads data from students.txt sequentially, providing clear visibility into the entire database

SEARCH MODULE

```
void searchStudent() {
    int roll;
    printf("Enter roll number to search: ");
    scanf("%d", &roll);

    struct student st;
    FILE *fp = fopen(STUDENT_FILE, "r");
    int found = 0;

    while (fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) != EOF) {
        if (st.roll == roll) {
            printf("\nFound: %d %s %.2f\n", st.roll, st.name, st.marks);
            found = 1;
            break;
        }
    }

    if (!found)
        printf("Student not found!\n");

    fclose(fp);
}
```

OUTPUT

```
===== ADMIN MENU =====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout
Enter choice: 3
Enter roll number to search: 2
Found: 2 AP400040375 30.00
```

The search module allows users to find a particular student record based on the roll number. It scans through the file line by line and prints the precise record if found. If no match exists, it informs the user accordingly.

UPDATE MODULE

```
void updateStudent() {
    int roll;
    printf("Enter roll number to update: ");
    scanf("%d", &roll);

    struct student st;
    FILE *fp = fopen(STUDENT_FILE, "r");
    FILE *tmp = fopen("temp.txt", "w");

    int found = 0;

    while (fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) != EOF) {
        if (st.roll == roll) {
            found = 1;
            printf("Enter new name: ");
            scanf("%s", st.name);
            printf("Enter new marks: ");
            scanf("%f", &st.marks);
        }
        fprintf(tmp, "%d %s %.2f\n", st.roll, st.name, st.marks);
    }

    fclose(fp);
    fclose(tmp);

    if (found) {
        remove(STUDENT_FILE);
        rename("temp.txt", STUDENT_FILE);
        printf("Record updated!\n");
    } else {
        remove("temp.txt");
        printf("Roll number not found!\n");
    }
}
```

OUTPUT

```
===== ADMIN MENU =====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout
Enter choice: 4
Enter roll number to update: 204
Enter new name: Sai
Enter new marks: 95.3
Record updated!
```

This module enables modifying an existing student's name and marks. It rewrites the target record with updated values while preserving the rest of the file using a temporary file mechanism. This ensures data consistency and clean updates.

DELETE MODULE

```
void deleteStudent() {
    int roll;
    printf("Enter roll number to delete: ");
    scanf("%d", &roll);

    struct student st;
    FILE *fp = fopen(STUDENT_FILE, "r");
    FILE *tmp = fopen("temp.txt", "w");

    int found = 0;

    while (fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) != EOF) {
        if (st.roll == roll) {
            found = 1;
            continue;
        }
        fprintf(tmp, "%d %s %.2f\n", st.roll, st.name, st.marks);
    }

    fclose(fp);
    fclose(tmp);

    if (found) {
        remove(STUDENT_FILE);
        rename("temp.txt", STUDENT_FILE);
        printf("Record deleted!\n");
    } else {
        remove("temp.txt");
        printf("Roll number not found!\n");
    }
}
```

OUTPUT

```
===== ADMIN MENU =====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout
Enter choice: 5
Enter roll number to delete: 204
Record deleted!
```

This module permanently removes a student record from the database based on the provided roll number. It copies all other records into a temporary file and replaces the original file. Only Admin users can access this module to prevent accidental data loss.